

BITÁCORA-PROCESO-CODIGO

En este trabajo realizado está, cómo pensé a el problema, las dificultades con las que me encontré, y cómo lo resolví, el aprendizaje que me dejó. Al principio lo pensé con un proceso recursivo, también usando mapcar y lambda para recorrer la lista usando las funciones recursivas lo pensé por como trabajan devolviendo resultados a través de los parámetros..

Luego cambie el enfoque porque se necesitaba una lista que devuelva algo. Comencé a trabajar cargando los datos con let,read, luego pude comprobar que no era necesario pedir al operador el ingreso de esas funciones, porque los valores se reciben por parámetros.

```
10 (transicion 'en-rojo 'verde) → ('en-rojo "cambiar-a-verde")
11
12
13 (defun actividad (color-actual cambiar-a)
14   (let ((color-actual(read))
15         (cambiar-a(read))
16         (transicion color-actual cambiar-a)
17       ))
18 )
19
20 (defun transicion (color-actual cambiar-a)
21   (cond
22     ((eq cambiar-a 'rojo);compara si dos simbolos son el mismo
23      (list color-actual "cambiar-a-rojo"));contruye la lista
24
25     ((eq cambiar-a 'amarillo)
26      (list color-actual "cambiar-a-amarillo"))
27
28     ((eq cambiar-a 'verde)
```

En este caso está hecho la corrección de quitar las funciones innecesarias

```
8 Comportamiento: Por defecto, retorna color actual y 'accion-por-defecto
9 Ejemplo esperado:
10 (transicion 'en-rojo 'verde) → ('en-rojo "cambiar-a-verde")
11
12
13 (defun actividad (color-actual cambiar-a)
14   (defun transicion (color-actual cambiar-a)
15     (cond
16       ((eq cambiar-a 'rojo);compara si dos simbolos son el mismo
17        (list color-actual "cambiar-a-rojo"));contruye la lista
18
19       ((eq cambiar-a 'amarillo)
20        (list color-actual "cambiar-a-amarillo"))
21
22       ((eq cambiar-a 'verde)
23        (list color-actual "cambiar-a-verde"))
24
25       (t
26        (list color-actual 'accion-por-defecto)))
27     )
28   )
29 )
```

Se incorporó las validaciones para garantizar que los datos recibidos correspondan a estados y colores válidos del semáforo. Antes de las funciones eq, se agregó la funciones or y not, permite comparar símbolos, verifica si el valor coincide con alguna de las opciones válidas y not invierte el resultado para detectar los casos incorrectos.

Las validaciones permiten aumentar la robustez del sistema, evitando que el semáforo procese estados o colores existentes y garantizando un comportamiento más seguro.

```
1
2
3
4 (defun transicion (color-actual cambiar-a)
5   (cond
6
7     ((not (or (eq color-actual 'en-rojo)
8               (eq color-actual 'en-amarillo)
9               (eq color-actual 'en-verde)))
10      (list 'error "color-actual-invalido"))
11
12     ((not (or (eq cambiar-a 'rojo)
13               (eq cambiar-a 'amarillo)
14               (eq cambiar-a 'verde)))
15      (list 'error "color-destino-invalido"))
16
17     ((eq cambiar-a 'rojo)
18      (list color-actual "cambiar-a-rojo"))
19
20     ((eq cambiar-a 'amarillo)
21      (list color-actual "cambiar-a-amarillo"))
22
23     ((eq cambiar-a 'verde)
24      (list color-actual "cambiar-a-verde"))
25
26     (t
27      (list color-actual "accion-por-defecto"))
```

```
"caso valido"
1 (transicion 'en-rojo 'verde)
2 (or (eq 'en-rojo 'en-rojo)
3     (eq 'en-rojo 'en-amarillo)
4     (eq 'en-rojo 'en-verde))
5 (or T NIL NIL)
6
7 (T
8   (not T)
9   (NIL) "si condicion NO se cumple y sigue")
10
11 "Caso Invalido"
12 (transicion 'azul 'verde)
13 (or (eq 'azul 'en-rojo)
14     (eq 'azul 'en-amarillo)
15     (eq 'azul 'en-verde))
16 (or NIL NIL NIL)
17 (NIL)
18 (not NIL T)
19 (list 'error "color-actual-invalido")
20 (ERROR "color-actual-invalido")
```